Intern Mobile Engineer Test

Part 1 - Build "TV App"

A simple TV show browser app with two screens, using the **TVMaze API**

- **List endpoint:** `GET https://api.tvmaze.com/shows?page=0`
- **Detail endpoint:** `GET https://api.tvmaze.com/shows/{id}`
- Docs: https://www.tvmaze.com/api

1. **List screen**
   Fetch shows from the list endpoint and display each show's **poster** (`image.medium`), **title** (`name`), and **rating** (`rating.average` — note: this can be `null`, handle it).

Loading one page (~250 shows) is enough, pagination is optional bonus, not required. The API is rate-limited to ~20 calls per 10 seconds, which you won't hit with normal use.

2. **Detail screen**
   Tapping an item shows the larger poster (`image.original`), title, **summary** (`summary` — note: it contains HTML tags like `<p>` and `<b>`, render or strip them properly), and **premiere date** (`premiered`).

Optional (bonus point) : showing the season, episode & cast.

3. **Share action**
   Share the TV show from Detail Screen. Content: title, summary and embed the url.

Requirements:

- **Android**: Kotlin + Jetpack Compose. **iOS**: Swift + SwiftUI.
- Handle three states: loading, error (with retry), success.
- Sensible architecture (e.g., MVVM). Structure over perfection.
- At least **2 unit tests** for your ViewModel or data layer.
- `README.md`: how to run, your architecture decisions, what you'd improve with more time.
- **Commit as you go** - we read your commit history to understand how you build. One giant final commit hurts your evaluation.

Spend no more than 1 day. We'd rather see 70% done well than 100% done messily.

Part 2 - AI Usage Log

AI tools (Claude, ChatGPT, Copilot, Cursor, etc.) are **allowed and encouraged**. We're evaluating how well you use AI, not whether you avoided it. Submit `AI_LOG.md` with **4–8 entries**, each covering:

1. What I asked the AI / the problem I was solving
2. What it gave me
3. What I did: accepted as-is / modified (how?) / rejected (why?)
4. One thing the AI got wrong or that I verified myself

Using AI without a log, or a log that says "asked AI, it worked" for every entry, counts against you. An honest log showing where AI failed counts **for** you.

Part 3 - AI Code Review Exercise (in writing)

An AI generated the code below. Imagine you're reviewing this PR. In a file called `CODE_REVIEW.md`, list every problem you'd flag and how you'd fix each one.

If applying for Android:

Kotlin

```kotlin
class MovieViewModel : ViewModel() {
    var movies: List<Movie> = emptyList()

    fun loadMovies() {
        val url = URL("https://api.example.com/movies")
        val data = url.readText()
        movies = parseMovies(data)
    }
}
```

If applying for iOS:

Swift

```swift
class MovieViewModel: ObservableObject {
    var movies: [Movie] = []

    func loadMovies() {
        let data = try! Data(contentsOf: URL(string:
"https://api.example.com/movies")!)
        movies = try! JSONDecoder().decode([Movie].self, from:
data)
    }
}
```

Part 4 - Written Reflection (short answers, in `REFLECTION.md`)

Answer honestly and specifically, there are no trick questions:

1. Which part of your submission are you **least** confident about, and why?
2. Describe a moment during this project (or any past project) where you got completely stuck. What did you do, step by step?
3. Imagine: it's Thursday, your task is due Friday, and you realize you misunderstood the requirement, half your work is wrong. What are you doing now?
4. Your mentor asks you to change an approach you believe is worse. What do you do?
5. What's something technical you taught yourself recently outside of class/work, and how did you learn it?

Part 5 - 5-Minute Walkthrough Video

Record your screen + voice (Loom, OBS, or phone — quality doesn't matter, no editing needed). In under 5 minutes:

1. Demo the app running (including the error state).
2. Walk through **one file you're proud of** and explain it line by line.
3. Show **one thing the AI got wrong** during the project and how you fixed it.

Submission

One GitHub repo (public or invite us) containing: code, `README.md`, `AI_LOG.md`, `CODE_REVIEW.md`, `REFLECTION.md`, and a link to the video in the README.